A. D. Patel Institute Of Technology
(a constituent of the CVM university)

University Name: CVM University

College Name: A. D. Patel Institute of Technology

Course Title: Programming with JAVA

Course Code: 202044502

Assignment No.: Assignment -1

Student Name: Mauli M Patel

Enrollment No.: 12502080603011

Semester: 4th semester

Academic year: 2025 – 26

Program – 1

Title: Implement Array and String operations (Reverse, Sort, Search, Average, Maximum) using class and objects

Objective:

To design and implement a Java class that performs various array and string operations such as reversing elements, sorting data, searching for a specific element, calculating average values, and identifying the maximum element using object-oriented programming concepts.

Source Code:

```
import java.util.*;

class ArrayOperations {

    int arr[];

    // Constructor

    ArrayOperations(int arr[]) {

        this.arr = arr;

    }

    // Reverse Array

    void reverse() {

        System.out.println("Reversed Array:");

        for (int i = arr.length - 1; i >= 0; i--) {

            System.out.print(arr[i] + " ");

        }

        System.out.println();

    }

    // Sort Array

    void sort() {

        Arrays.sort(arr);

        System.out.println("Sorted Array:");
```

```
        for (int i : arr) {
            System.out.print(i + " ");
        }
        System.out.println();
    }
    // Search Element
    void search(int key) {
        boolean found = false;
        for (int i : arr) {
            if (i == key) {
                found = true;
                break;
            }
        }
        if (found){
            System.out.println("Element Found");
}
        else{
            System.out.println("Element Not Found");
    }
    // Average
    void average() {
        int sum = 0;
        for (int i : arr)
            sum += i;
        double avg = (double) sum / arr.length;
        System.out.println("Average = " + avg);
    }
```

```
    // Maximum
    void maximum() {
        int max = arr[0];
        for (int i : arr) {
            if (i > max)
                max = i;
        }
        System.out.println("Maximum = " + max);
    }
}
class Main1 {
    public static void main(String args[]) {
        int arr[] = {5, 2, 9, 1, 6};
        ArrayOperations obj = new ArrayOperations(arr);
        obj.reverse();
        obj.sort();
        obj.search(9);
        obj.average();
        obj.maximum();
    }
}
```

Output:

```
Reversed Array:
6 1 9 2 5
Sorted Array:
1 2 5 6 9
Element Found
Average = 4.6
Maximum = 9

Process finished with exit code 0
```

Program – 2

Aim: Develop Matrix class with constructors, transpose and multiplication

Objective:
To develop a Matrix class in Java using constructors and implement matrix operations such as transpose and matrix multiplication, demonstrating the use of two-dimensional arrays and object-oriented programming principles.

Source Code:

```
class Matrix {
    int m[][] = new int[2][2];
    // Constructor
    Matrix(int a, int b, int c, int d) {
        m[0][0] = a;
        m[0][1] = b;
        m[1][0] = c;
        m[1][1] = d;
    }
    // Transpose
    void transpose() {
        System.out.println("Transpose:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                System.out.print(m[j][i] + " ");
            }
            System.out.println();
        }
    }
```

```
    // Multiplication
    void multiply(Matrix m2) {

        int result[][] = new int[2][2];

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {

                result[i][j] = 0;

                for (int k = 0; k < 2; k++) {
                    result[i][j] += m[i][k] * m2.m[k][j];
                }
            }
        }
        System.out.println("Multiplication Result:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                System.out.print(result[i][j] + " ");
            }
            System.out.println();
        }
    }
}
class Main2 {
    public static void main(String args[]) {
        Matrix m1 = new Matrix(1, 2, 3, 4);
        Matrix m2 = new Matrix(5, 6, 7, 8);
```

```
        m1.transpose();
        m1.multiply(m2);
    }
}
```

Output:

```
Transpose:
1 3
2 4
Multiplication Result:
19 22
43 50

Process finished with exit code 0
```

Program - 3

Aim: Write a program demonstrating Wrapper classes and String vs StringBuffer

Objective:
To demonstrate the use of wrapper classes for converting primitive data types into objects and to compare the behavior of String and StringBuffer classes in terms of mutability and performance in Java.

Source Code:

```
class WrapperDemo {
    void wrapperExample() {
        int num = 10;
        // Primitive to Object
        Integer obj = Integer.valueOf(num);
        // Object to Primitive
        int value = obj.intValue();
        System.out.println("Wrapper Object = " + obj);
        System.out.println("Primitive Value = " + value);
    }
    void stringExample() {
        String s1 = "Hello";
        s1.concat(" World");
        System.out.println("String (Immutable): " + s1);
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World");
        System.out.println("StringBuffer (Mutable): " + sb);
    }
}
class Main3 {
```

```
    public static void main(String args[]) {
        WrapperDemo obj = new WrapperDemo();
        obj.wrapperExample();
        obj.stringExample();
    }
}
```

Output:

```
Wrapper Object = 10
Primitive Value = 10
String (Immutable): Hello
StringBuffer (Mutable): Hello World

Process finished with exit code 0
```

Program – 4

Aim: Implement BankAccount class with deposit, withdraw and balance inquiry

Objective:
To design and implement a BankAccount class that performs basic banking operations such as depositing money, withdrawing funds, and checking account balance, illustrating the concept of encapsulation and method implementation.

Source Code:

```
class BankAccount {
   int balance = 1000;
   void deposit(int amount) {
      balance += amount;
      System.out.println("Deposited: " + amount);
   }
   void withdraw(int amount) {
      if (amount <= balance) {
         balance -= amount;
         System.out.println("Withdrawn: " + amount);
      } else {
         System.out.println("Insufficient Balance");
      }
   }
   void checkBalance() {
      System.out.println("Balance = " + balance);
   }
}
class Main4 {
```

```
    public static void main(String args[]) {
        BankAccount obj = new BankAccount();
        obj.deposit(500);
        obj.withdraw(300);
        obj.checkBalance();
    }
}
```

Output:

```
Deposited: 500
Withdrawn: 300
Balance = 1200

Process finished with exit code 0
```

Program - 5

Aim: Develop inheritance-based Cricket–Match system using command line arguments

Objective:
To implement inheritance in Java by developing a Cricket Match system that accepts command-line arguments and demonstrates the relationship between base and derived classes.

Source Code:

```
import java.util.Scanner;
class Cricket {
   String team1, team2;
   Cricket(String t1, String t2) {
      team1 = t1;
      team2 = t2;
   }
   void showTeams() {
      System.out.println("Match Between: " + team1 + " vs " + team2);
   }
}
class Match extends Cricket {
   int score1, score2;
   Match(String t1, String t2, int s1, int s2) {
      super(t1, t2);
      score1 = s1;
      score2 = s2;
   }
   void result() {
      if (score1 > score2)
         System.out.println(team1 + " wins!");
      else if (score2 > score1)
         System.out.println(team2 + " wins!");
      else
         System.out.println("Match Draw!");
   }
   void display() {
      showTeams();
      System.out.println(team1 + " Score: " + score1);
      System.out.println(team2 + " Score: " + score2);
      result();
   }
```

```
}
public class Program_5
{
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter Team 1: ");
      String t1 = sc.nextLine();
      System.out.print("Enter Team 2: ");
      String t2 = sc.nextLine();
      System.out.print("Enter Score of " + t1 + ": ");
      int s1 = sc.nextInt();
      System.out.print("Enter Score of " + t2 + ": ");
      int s2 = sc.nextInt();
      Match m = new Match(t1, t2, s1, s2);
      m.display();
   }
}
```

Output:

```
Enter Team 1: India
Enter Team 2: Pakisthan
Enter Score of India: 253
Enter Score of Pakisthan: 182
Match Between: India vs Pakisthan
India Score: 253
Pakisthan Score: 182
India wins!
```

Program - 6

Aim: Implement Cipher system using Abstract class and method overriding

Objective:
To implement a Cipher system using an abstract class and demonstrate method overriding by creating different encryption methods in derived classes.

Source Code:

```
abstract class Cipher {
   abstract void encrypt(String text);
}
class CaesarCipher extends Cipher {
   void encrypt(String text) {
      String result = "";
      for (int i = 0; i < text.length(); i++) {
         char ch = text.charAt(i);
         ch += 3;
         result += ch;
      }
      System.out.println("Encrypted Text: " + result);
   }
}
class Main6 {
   public static void main(String args[]) {
      CaesarCipher obj = new CaesarCipher();
      obj.encrypt("HELLO");
   }
}
```

Output:

```
Encrypted Text: KHOOR

Process finished with exit code 0
```

Program - 7

Aim: Demonstrate Inner Classes (member, local, anonymous)

Objective:
To demonstrate the use of different types of inner classes in Java, including member inner class, local inner class, and anonymous inner class, and understand their applications in program design.

Source Code:

```
class Outer {
    int x = 10;
    // Member Inner Class
    class MemberInner {
        void show() {
            System.out.println("Member Inner Class: " + x);
        }
    }
    void display() {
        // Local Inner Class
        class LocalInner {
            void show() {
                System.out.println("Local Inner Class");
            }
        }
        LocalInner li = new LocalInner();
        li.show();
        // Anonymous Inner Class
        Runnable r = new Runnable() {
            public void run() {
```

```
            System.out.println("Anonymous Inner Class");
        }
    };
    r.run();
  }
}
class Main7 {
  public static void main(String args[]) {
    Outer obj = new Outer();
    Outer.MemberInner mi = obj.new MemberInner();
    mi.show();
    obj.display();
  }
}
```

Output:

```
Member Inner Class: 10
Local Inner Class
Anonymous Inner Class

Process finished with exit code 0
```

Program - 8

Aim: Create custom exception handling for bank withdrawal scenario

Objective:
To create and implement a custom exception class in Java to handle exceptional situations such as insufficient balance during withdrawal, demonstrating the concept of exception handling and error management.

Source Code:

```
class InsufficientBalanceException extends Exception {
   InsufficientBalanceException(String message) {
      super(message);
   }
}
class Bank {
   int balance = 1000;
   void withdraw(int amount) throws InsufficientBalanceException {
      if (amount > balance) {
         throw new InsufficientBalanceException("Insufficient Balance!");
      } else {
         balance -= amount;
         System.out.println("Withdrawn: " + amount);
      }
   }
}
class Main8 {
   public static void main(String args[]) {
      Bank obj = new Bank();
      try {
```

```
        obj.withdraw(1500);
    } catch (InsufficientBalanceException e) {
        System.out.println(e.getMessage());
    }
  }
}
```

Output:

```
Insufficient Balance!

Process finished with exit code 0
```